National School of Cyber Security (NSCS)
Department of Foundation Training | OOP with Java

Technical Documentation

ENSCS Internship Management System

Final Project — Object-Oriented Programming with Java
Full-Stack Web Application

Technology Stack
Backend: Spring Boot 3 / Java 17
Frontend: React 18 + TypeScript
Database: PostgreSQL + JPA/Hibernate
Auth: JWT + Spring Security

Project Info
Module: Object-Oriented Programming
Project: ENSCS Internship Management
Submitted: May 2026
School: NSCS Algeria

1. Project Overview

The ENSCS Internship Management System is a full-stack web application built to digitalise and streamline the internship lifecycle at the National School of Cyber Security. It replaces paper-based workflows with a centralised platform where students can discover and apply for internship offers, supervisors can evaluate progress, and administrators manage the entire pipeline.

1.1 System Purpose

> Core goal: provide a single, role-aware platform that covers every stage of an internship — from offer publication, through student application and selection, to supervisor evaluation and final report submission.

1.2 User Roles

Role	Access Level	Key Responsibilities
Student	Limited	Browse offers, submit applications, upload reports, view evaluations
Supervisor	Intermediate	Publish/manage internship offers, evaluate assigned students, write reports
Admin	Full	Manage all users, approve/reject offers, configure platform, view analytics

1.3 Key Features

- Internship offer management — CRUD operations with approval workflow
- Student application system — apply, track status, receive notifications
- Supervisor evaluation — structured grading form with criteria scoring
- File upload — students submit internship reports (PDF/DOCX)
- Role-based dashboards — tailored views and data for each role
- JWT authentication — secure, stateless login with refresh tokens
- RESTful API — documented with OpenAPI / Swagger UI

2. System Architecture

2.1 High-Level Architecture

The system follows a classic three-tier architecture with a React SPA on the client, a Spring Boot REST API server in the middle, and a PostgreSQL database at the persistence layer.

```
[ React 18 + TypeScript SPA ]
        ↕ HTTPS / JSON (REST API)
[ Spring Boot 3 REST API — Java 17 ]
        ↕ JPA / Hibernate
[ PostgreSQL Database ]
```

2.2 Backend Architecture — Spring Boot

The backend is structured around a layered package architecture, ensuring strict separation of concerns:

Layer / Package	Responsibility
controller	HTTP endpoints — receives requests, delegates to services, returns responses
service	Business logic — orchestrates domain rules, calls repositories
repository	Data access — Spring Data JPA interfaces extending JpaRepository
model / entity	JPA entities mapped to DB tables; inheritance hierarchy with @Inheritance
dto	Data Transfer Objects — decouple API contracts from entity structure
security	Spring Security config, JWT filter chain, UserDetails implementation
exception	Custom exceptions + @ControllerAdvice global error handler
util	Shared utilities — file storage helpers, validators, mappers

2.3 Frontend Architecture — React + TypeScript

Concern	Technology / Approach
State management	Zustand stores (auth, UI state)
Server state & caching	React Query (TanStack Query)
Routing	React Router v6 with protected routes
Forms & validation	React Hook Form + Zod schema validation

Styling	Tailwind CSS with inline utility classes
Charts / analytics	Recharts
HTTP client	Axios with JWT interceptors
Type safety	TypeScript strict mode throughout

3. Object-Oriented Programming Principles

This section is the heart of the technical report. Each OOP principle is explained in the context of how it is concretely applied in the project — primarily in the Spring Boot backend.

3.1 Encapsulation

Encapsulation bundles data and the methods that operate on it inside a class, exposing only what is necessary through a controlled interface.

How it is applied in the backend

- All JPA entity fields are private. Public access is provided exclusively through getters and setters (or Lombok @Getter / @Setter).
- The service layer owns business logic. Controllers never manipulate entity state directly — they receive DTOs, pass them to services, and return response DTOs.
- The repository layer is the only place that touches the database; services cannot bypass it.

Code example — Internship entity

```
@Entity
public class Internship {
    @Id @GeneratedValue
    private Long id;
    private String title;
    private String company;
    private InternshipStatus status;   // private field

    public void publish() {             // controlled state change
        if (this.status != InternshipStatus.DRAFT)
            throw new IllegalStateException("Only drafts can be published");
        this.status = InternshipStatus.OPEN;
    }
    // getters / setters ...
}
```

The status field cannot be set arbitrarily from outside — the publish() method enforces the transition rule. This is encapsulation of both data and behaviour.

3.2 Inheritance

Inheritance allows a child class to reuse and extend the state and behaviour of a parent class, avoiding code duplication and establishing IS-A relationships.

User hierarchy — JPA @Inheritance

All user types share common identity and authentication data. A User base entity holds the shared fields; Student, Supervisor, and Admin extend it.

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class User {
    @Id @GeneratedValue
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private String password;
    @Enumerated(EnumType.STRING)
    private Role role;
}
```

```
@Entity
public class Student extends User {
    private String studentId;
    private String department;
    private int year;
    @OneToMany(mappedBy = "student")
    private List<Application> applications;
}
```

```
@Entity
public class Supervisor extends User {
    private String organization;
    private String specialization;
    @OneToMany(mappedBy = "supervisor")
    private List<Internship> offeredInternships;
}
```

The JOINED strategy maps each subclass to its own table. Common columns live in the user table, specific columns in student / supervisor. Spring Security loads any user via UserDetailsService regardless of the concrete subtype.

BaseEntity for audit fields

A BaseEntity abstract class provides createdAt / updatedAt timestamps via @MappedSuperclass so every entity inherits audit support without repeating the fields:

```
@MappedSuperclass
public abstract class BaseEntity {
    @CreatedDate
    private LocalDateTime createdAt;
    @LastModifiedDate
    private LocalDateTime updatedAt;
}
```

3.3 Polymorphism

Polymorphism allows the same method call to behave differently depending on the actual runtime type of the object. The project uses both runtime (dynamic) and compile-time (static/overloading) polymorphism.

Runtime polymorphism — Evaluatable interface

Different entities that can be evaluated (Internship, Application) share a common contract. The evaluation service works against the interface, not concrete types:

```
public interface Evaluatable {
    double calculateScore();
    EvaluationStatus getEvaluationStatus();
    void submitEvaluation(EvaluationDTO dto);
}
```

```
@Entity
public class Application implements Evaluatable {
    @Override
    public double calculateScore() {
        return criteria.stream()
            .mapToDouble(c -> c.getWeight() * c.getScore())
            .sum() / 100.0;
    }
    // ...
}
```

Polymorphism via Spring's @Service

Spring's dependency injection enables polymorphism at the service level. A NotificationService interface has multiple implementations (EmailNotificationService, InAppNotificationService). The calling code depends only on the interface:

```
public interface NotificationService {
    void notify(User recipient, String message);
}
```

```
// Spring injects the correct bean at runtime
@Autowired
private NotificationService notificationService;
```

Method overriding — UserDetailsService

The CustomUserDetailsService class overrides loadUserByUsername() from Spring Security's interface. Depending on the actual User subtype retrieved, the returned UserDetails object carries different authorities — demonstrating behavioural polymorphism:

```
@Service
public class CustomUserDetailsService implements UserDetailsService {
    @Override
    public UserDetails loadUserByUsername(String email) {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException(email));
        return new org.springframework.security.core.userdetails.User(
            user.getEmail(), user.getPassword(),
            List.of(new SimpleGrantedAuthority("ROLE_" + user.getRole())));
    }
}
```

3.4 Abstraction

Abstraction hides implementation complexity and exposes only the essential behaviour. In Java, abstraction is achieved through abstract classes and interfaces.

Service interfaces

Every major service is defined as an interface (InternshipService, ApplicationService, UserService). The controller layer depends on interfaces only; the concrete implementation classes are injected by Spring:

```
public interface InternshipService {
    Page<InternshipDTO> getOpenOffers(Pageable pageable);
    InternshipDTO createOffer(CreateInternshipRequest request, Long supervisorId);
    void approveOffer(Long internshipId);
    void rejectOffer(Long internshipId, String reason);
}
```

A developer reading the controller does not need to know how approveOffer() works internally. The interface is the contract; the implementation detail is hidden.

Abstract Report class

Different report types (InternshipReport, ProgressReport) share a common abstract base that defines the report lifecycle. Concrete subclasses fill in type-specific behaviour:

```
public abstract class Report extends BaseEntity {
    protected String title;
    protected String filePath;
    protected ReportStatus status;

    public abstract String getReportType();
    public abstract boolean isComplete();

    public void submit() {
        if (!isComplete()) throw new IncompleteReportException();
```

```
        this.status = ReportStatus.SUBMITTED;
    }
}
```

3.5 Interfaces

Java interfaces define contracts that multiple unrelated classes can fulfil. The project uses several custom interfaces beyond the Spring framework ones.

Interface	Implementing class(es)	Purpose
Evaluatable	Application, Internship	Standardise evaluation scoring
Submittable	Application, Report	Enforce submit/withdraw lifecycle
NotificationService	EmailNotification, InAppNotification	Swap notification channel
FileStorageService	LocalFileStorage, S3FileStorage	Abstract file upload target
Reportable	InternshipReport, ProgressReport	Common report operations

Using interfaces rather than abstract classes where appropriate keeps coupling low — a class can implement multiple interfaces (e.g., Application implements both Evaluatable and Submittable) but can extend only one class.

3.6 Exception Handling & Custom Exceptions

The project defines a hierarchy of custom checked and unchecked exceptions that map cleanly onto HTTP status codes. A @ControllerAdvice class intercepts them globally.

Custom exception hierarchy

```
// Base application exception
public class AppException extends RuntimeException {
    private final HttpStatus status;
    public AppException(String message, HttpStatus status) {
        super(message);
        this.status = status;
    }
    public HttpStatus getStatus() { return status; }
}
```

```
public class ResourceNotFoundException extends AppException {
    public ResourceNotFoundException(String resource, Long id) {
        super(resource + " not found with id: " + id, HttpStatus.NOT_FOUND);
    }
```

```
}
```

```
public class DuplicateApplicationException extends AppException {
    public DuplicateApplicationException() {
        super("Student already applied to this internship", HttpStatus.CONFLICT);
    }
}
```

```
public class IncompleteReportException extends AppException {
    public IncompleteReportException() {
        super("Report must be complete before submission",
HttpStatus.UNPROCESSABLE_ENTITY);
    }
}
```

Global exception handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(AppException.class)
    public ResponseEntity<ErrorResponse> handleApp(AppException ex) {
        return ResponseEntity.status(ex.getStatus())
            .body(new ErrorResponse(ex.getMessage(), ex.getStatus().value()));
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
            .map(e -> e.getField() + ": " + e.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest().body(new ErrorResponse(msg, 400));
    }
}
```

3.7 Java Collections Framework

The Java Collections Framework is used throughout the service layer for aggregation, filtering, sorting, and transformation of data.

Collection type	Where used in the project
List<Application>	All applications for a given internship offer
List<Internship>	Supervisor's published offers
Map<String, Long>	Analytics: count per application status
Set<String>	Unique file extensions allowed for upload validation
Optional<User>	Repository lookup — avoids null checks

Stream API example — dashboard statistics

```
public DashboardStatsDTO getAdminStats() {
    List<Application> all = applicationRepository.findAll();

    Map<ApplicationStatus, Long> countByStatus = all.stream()
        .collect(Collectors.groupingBy(
            Application::getStatus, Collectors.counting()));

    double avgScore = all.stream()
        .filter(a -> a.getEvaluationStatus() == EvaluationStatus.DONE)
        .mapToDouble(Application::calculateScore)
        .average().orElse(0.0);

    return new DashboardStatsDTO(countByStatus, avgScore);
}
```

3.8 File Storage (I/O)

Students upload internship reports. The backend handles multipart file uploads, validates extensions, stores files to a configurable path, and returns download links.

```
@Service
public class LocalFileStorageService implements FileStorageService {
    private final Path storageRoot;
    private static final Set<String> ALLOWED =
        Set.of("pdf", "doc", "docx");

    @Override
    public String store(MultipartFile file, Long studentId) {
        String ext = getExtension(file.getOriginalFilename());
        if (!ALLOWED.contains(ext))
            throw new InvalidFileTypeException(ext);

        String filename = studentId + "_" + UUID.randomUUID() + "." + ext;
        Path target = storageRoot.resolve(filename);
        try { Files.copy(file.getInputStream(), target); }
        catch (IOException e) { throw new FileStorageException(e); }
        return filename;
    }
}
```

4. Backend Deep Dive

4.1 Entity Model & Relationships

The data model centres on six core entities. The table below summarises each entity and its primary relationships.

Entity	Extends / Implements	Key Relationships
User	BaseEntity	Abstract parent of Student, Supervisor, Admin
Student	User	@OneToMany → Application, @OneToMany → Report
Supervisor	User	@OneToMany → Internship
Internship	BaseEntity	@ManyToOne → Supervisor, @OneToMany → Application
Application	BaseEntity	@ManyToOne → Student, @ManyToOne → Internship
Report	BaseEntity (abstract)	Extended by InternshipReport, ProgressReport

4.2 Security — JWT Authentication

- Spring Security filter chain intercepts every request.
- JwtAuthenticationFilter extracts and validates the Bearer token from the Authorization header.
- On success, it sets the SecurityContextHolder with the authenticated user's authorities.
- Role-based endpoint protection via @PreAuthorize annotations (e.g., @PreAuthorize("hasRole('ADMIN')")).
- Refresh token mechanism to extend sessions without re-login.

4.3 REST API Design

The API follows REST conventions. All endpoints are versioned under /api/v1/ and documented via SpringDoc OpenAPI.

Method + Path	Role required	Description
GET /api/v1/internships	PUBLIC	List open internship offers (paginated)
POST /api/v1/internships	SUPERVISOR	Create a new offer
POST /api/v1/internships/{id}/approve	ADMIN	Approve a pending offer
POST /api/v1/applications	STUDENT	Submit an application
GET /api/v1/applications/my	STUDENT	Get own applications

POST /api/v1/applications/{id}/evaluate	SUPERVISOR	Submit evaluation
POST /api/v1/reports/upload	STUDENT	Upload internship report file
GET /api/v1/admin/stats	ADMIN	Platform-wide statistics

4.4 Modular Package Structure

```
src/main/java/com/enscs/internship/
├── controller/
│   ├── AuthController.java
│   ├── InternshipController.java
│   ├── ApplicationController.java
│   ├── ReportController.java
│   └── AdminController.java
├── service/
│   ├── InternshipService.java          (interface)
│   ├── InternshipServiceImpl.java
│   ├── ApplicationService.java         (interface)
│   ├── ApplicationServiceImpl.java
│   ├── FileStorageService.java          (interface)
│   └── LocalFileStorageServiceImpl.java
├── repository/
│   ├── UserRepository.java
│   ├── InternshipRepository.java
│   └── ApplicationRepository.java
├── model/
│   ├── User.java  (abstract, @Inheritance JOINED)
│   ├── Student.java
│   ├── Supervisor.java
│   ├── Admin.java
│   ├── Internship.java
│   ├── Application.java
│   └── report/
│       ├── Report.java  (abstract)
│       ├── InternshipReport.java
│       └── ProgressReport.java
├── dto/
├── security/
│   ├── JwtAuthenticationFilter.java
│   ├── JwtService.java
│   └── CustomUserDetailsService.java
└── exception/
    ├── AppException.java
    ├── ResourceNotFoundException.java
    ├── DuplicateApplicationException.java
    └── GlobalExceptionHandler.java
```

5. Frontend Deep Dive

5.1 Project Structure

```
src/
├── api/                    # Axios instances & API call functions
├── components/
│   ├── common/             # Shared UI: Button, Input, Modal, Badge
│   ├── layout/             # Sidebar, Topbar, PageWrapper
│   └── forms/              # InternshipForm, ApplicationForm, EvaluationForm
├── pages/
│   ├── auth/               # Login, Register
│   ├── student/            # Dashboard, Browse, Applications, Reports
│   ├── supervisor/         # Dashboard, MyOffers, Evaluations
│   └── admin/              # Dashboard, Users, Offers, Stats
├── stores/                 # Zustand: authStore, uiStore
├── hooks/                  # Custom hooks wrapping React Query
├── types/                  # TypeScript interfaces & enums
└── utils/                  # Validators, formatters, constants
```

5.2 Authentication Flow

- User logs in → POST /api/v1/auth/login → receives access token + refresh token.
- authStore (Zustand) persists the token and decoded user profile.
- Axios interceptor attaches Bearer token to every request automatically.
- On 401 response, interceptor calls the refresh endpoint; on failure, redirects to login.
- Protected routes check the store; unauthenticated users are redirected.

5.3 Role-Based UI

A single ProtectedRoute wrapper reads the user role from authStore and renders either the requested page or a 403 Forbidden component. Each role sees a completely different sidebar navigation and dashboard content:

Role	Sidebar links visible
Student	Home · Browse Internships · My Applications · My Reports
Supervisor	Home · My Offers · Create Offer · Evaluations
Admin	Dashboard · Users · Internship Offers · Analytics

5.4 Data Fetching with React Query

React Query manages all server state: caching, background refresh, loading/error states, and cache invalidation after mutations.

```
// hooks/useInternships.ts
```

```
export function useOpenInternships(page: number) {
  return useQuery({
    queryKey: ['internships', 'open', page],
    queryFn: () => api.get(`/internships?page=${page}`).then(r => r.data),
    staleTime: 60_000,
  });
}
```

```
export function useApplyMutation() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (internshipId: number) =>
      api.post('/applications', { internshipId }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['applications'] }),
  });
}
```

5.5 Form Validation — React Hook Form + Zod

```
const schema = z.object({
  title:       z.string().min(5, 'Title must be at least 5 characters'),
  company:     z.string().min(2),
  description: z.string().min(20),
  duration:    z.number().min(1).max(24),
  deadline:    z.string().refine(d => new Date(d) > new Date(), {
    message: 'Deadline must be in the future',
  }),
});
```

Zod schemas serve as the single source of truth for validation — they are shared between the form and the TypeScript type inference (z.infer<typeof schema>), eliminating duplication.

6. OOP Principles — Summary Checklist

The table below provides a quick reference mapping every required OOP concept from the project brief to a concrete implementation in the codebase.

OOP Concept	Requirement	Implementation
Encapsulation	Private fields + controlled access	All entity fields private; business rules enforced in methods (e.g., publish(), submit())
Inheritance	At least 1 hierarchy	User → Student / Supervisor / Admin (@Inheritance JOINED); BaseEntity @MappedSuperclass; Report (abstract) → InternshipReport / ProgressReport
Polymorphism	Method overriding / overloading	UserDetailsService.loadUserByUsername(), Evaluatable.calculateScore(), NotificationService polymorphic dispatch
Abstraction	Abstract classes & interfaces	Abstract User, abstract Report; service interfaces (InternshipService, FileStorageService)
Interfaces	At least 2 custom interfaces	Evaluatable, Submittable, NotificationService, FileStorageService, Reportable
Exception handling	try/catch + custom exceptions	AppException hierarchy; @RestControllerAdvice GlobalExceptionHandler
Custom exceptions	At least 2 custom exception classes	ResourceNotFoundException, DuplicateApplicationException, IncompleteReportException, InvalidFileTypeException, FileStorageException
Collections Framework	Multiple collection types	List, Map, Set, Optional throughout service layer; Stream API for aggregation
File I/O	Read/write files	LocalFileStorageService — Files.copy() for uploads; Files.readAllBytes() for downloads
Graphical interface	Swing / JavaFX or Web UI	React 18 SPA with role-based dashboards, forms, charts (Recharts)
Data persistence	Store data across sessions	PostgreSQL via JPA/Hibernate; Spring Data JPA repositories
Modular architecture	Clear package separation	controller / service / repository / model / dto / security / exception — strict layering
At least 8 classes	Class count ≥ 8	User, Student, Supervisor, Admin, Internship, Application, Report, InternshipReport, ProgressReport, JwtService, GlobalExceptionHandler… (20+)

7. UML Class Diagram Overview

A full UML class diagram is provided as a separate deliverable. The key relationships are summarised textually below.

```
BaseEntity ◄——— User (abstract) ◄——— Student
                                ◄——— Supervisor
                                ◄——— Admin

Student  1 ——— * Application  * ——— 1  Internship  * ——— 1  Supervisor

BaseEntity ◄——— Report (abstract) ◄——— InternshipReport
                                  ◄——— ProgressReport

Application  .implements.  Evaluatable
Application  .implements.  Submittable
Report       .implements.  Reportable
```

Inheritance depth

- Level 0 (root): BaseEntity / Object
- Level 1: User (abstract), Report (abstract)
- Level 2: Student, Supervisor, Admin, InternshipReport, ProgressReport

Interface implementation count

- 5 custom interfaces defined
- 7 implementing classes across the model and service layers

8. Conclusion

The ENSCS Internship Management System successfully demonstrates all eleven OOP principles required by the project brief. The backend, built with Spring Boot 3 and Java 17, serves as the primary showcase:

- Encapsulation: every domain rule (status transitions, submission checks) is enforced inside the owning class.
- Inheritance: the JOINED table-per-subclass strategy maps naturally to the user role hierarchy.
- Polymorphism: interface-driven design allows runtime swapping of notification channels and file storage targets.
- Abstraction: service interfaces decouple the API layer from the implementation layer completely.
- Exception handling: a typed hierarchy of custom exceptions maps to meaningful HTTP responses, improving API usability.
- Collections & Streams: rich data processing in the service layer without raw loops.
- File I/O: secure multipart upload with extension validation and UUID-based naming to prevent conflicts.

The React + TypeScript frontend adds a modern, type-safe presentation layer with role-specific dashboards, validated forms, and reactive server-state management — bringing the system to production-ready quality.

> This project goes beyond the minimum requirements: it applies OOP principles not only in isolation but in combination — e.g., polymorphism and abstraction together via interface-driven Spring services — reflecting how professional Java software is structured in the real world.